

TokenPowerSandbox: Evidence-Gated Multi-Fidelity Screening for Energy-Efficient LLM Inference on HPC Clusters

Anonymous Authors

Abstract—GPU allocations are scarce on HPC clusters, yet screening every LLM inference scenario on its target accelerator is costly. CPU-resident models can explore workloads without reserving a GPU, but their estimates are not hardware evidence. We present TokenPowerSandbox, an evidence-gated, multi-fidelity sandbox that couples an interpretable CPU projector and sparse measured correction with a real-GPU executor and freeze-before-measurement validation. On one H100 80GB serving Qwen2.5-7B with vLLM, a three-repeat anchor and six development workloads calibrate transfer across context length and concurrency. The same frozen model is then evaluated on an eight-workload blind holdout and, without refitting, a nine-workload preregistered scope confirmation—51 real H100 runs in total. Energy MAPE is 6.23% and 7.35% in the two studies. The confirmation passes its 15% energy endpoint; a preregistered TTFT rule passes at concurrency four (9.27% MAPE) but triggers abstention below four (64.80%). Thus CPU-first screening can preserve energy decisions, while evidence gating detects when performance fidelity does not transfer.

Index Terms—LLM inference, energy efficiency, workload modeling, multi-fidelity evidence, HPC clusters, GPU measurement

I. INTRODUCTION

HPC experiments have asymmetric resource cost: CPU allocations can execute deterministic projections without reserving accelerators, while GPU nodes may be scarce and queue-delayed. LLM serving behavior changes with token lengths, concurrency, arrival rate, batching, KV-cache controls, power, and tensor/pipeline parallelism (TP/PP). These factors jointly determine energy, throughput, time to first token (TTFT), and time per output token (TPOT). CPU-first screening avoids exhaustive target-GPU measurement, but risks mistaking a plausible projection for hardware evidence.

Measurement systems establish what a completed run consumed [1], [2], [3]; TokenPowerBench adds phase-aligned device and node telemetry across LLM inference configurations [4]. Serving simulators and learned recommendation systems reduce live experimentation [5], [6], [7]. Pre-deployment energy modeling nevertheless has a separate trust question: does a cheap model preserve the energy-SLO ordering that drives the next experiment, and can its failure be detected before deployment?

We ask: *how can CPU-accessible projections and sparse real measurements support auditable screening of an unseen LLM inference workload?* TokenPowerSandbox is not a Docker image or a set of virtual GPUs. It is a multi-fidelity workflow comprising a CPU-resident analytical model, a restricted real-GPU executor, and an evidence gate. Docker isolates and pins measured execution; the sandbox is the larger HPC resource and evidence contract shown in Fig. 1.

This WIP makes three bounded contributions:

- A CPU-first, GPU-verified evidence contract that separates workload from deployment controls and rejects comparisons with mismatched provenance or chronology.
- An interpretable workload-transfer model that combines analytical compute, memory, and power terms with a four-feature log-residual fitted only on development workloads.
- Two sealed H100/Qwen2.5-7B holdouts with 51 real measurements: 6.23% and 7.35% energy MAPE, plus a preregistered decision that releases TTFT at concurrency four and abstains below it.

The contribution is the evidence substrate, not yet an LLM agent, a GPU-queue scheduler, or a verified TP/PP simulator. Search policies can consume it only where the measured scope gate permits.

II. EVIDENCE CONTRACT

The sandbox compiles two disjoint objects. A *scenario* s freezes model and tokenizer revisions, token lengths, request count, arrival regime, target hardware, and SLO. A *configuration* $x \in \mathcal{X}$ contains only deployable controls: runtime, precision, batching and cache limits, TP/PP/DP, placement, and power cap. This prevents a search policy from reporting an “improvement” by silently reducing the requested work.

Every backend returns an append-only record

$$e = (h(s), h(x), \ell, k, \mathbf{y}, u, c, z), \quad (1)$$

where $h(\cdot)$ is a SHA-256 identity, ℓ is fidelity, k distinguishes predicted and measured records, $\mathbf{y}$ holds energy and SLO metrics, u is declared uncertainty, c is measured GPU-hour cost, and z records software, topology, telemetry, and failure provenance. L0 is a deterministic CPU projection; L1 is a phase-controlled single-GPU measurement. A topology-aware CPU projection remains *extrapolated* and retains *validation_required*. Zero incremental GPU-hours is not zero cost: CPU time, memory, and energy remain reportable overheads.

Predictions, intervals, and manifests are frozen before a holdout. The validator checks timestamps, hashes, split labels, scenario/configuration, container identity, power readback, repeat count, and every raw artifact. A comparison is rejected if any check fails. The present study exercises L1 calibration, L0 CPU prediction, and independent L1 validation; it makes no topology-aware or target-scale accuracy claim.

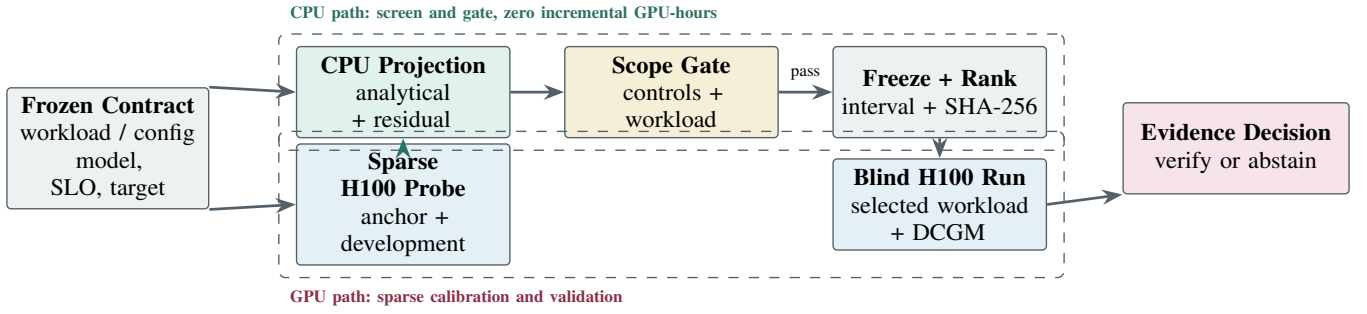


Fig. 1. TokenPowerSandbox separates HPC resource paths. CPU-resident projections screen candidates without reserving a target GPU; selected H100 runs calibrate and validate them. Prediction becomes evidence only through the release gate.

III. TOKENPOWERSANDBOX

A. CPU-Resident Analytical Core

The base projector rejects memory-infeasible candidates, then scales a measured anchor using token geometry, batch saturation, and declared topology. It separates prefill and decode, estimates active power between measured idle and active bounds, and derives device energy as $\hat{E} = \hat{P}t$. Its output includes energy per 1,000 output tokens, throughput, TTFT, TPOT, and a term-by-term decomposition.

One anchor cannot learn utilization transitions. We therefore fit a small, auditable correction after a development campaign. Relative to anchor workload (L_0, k_0) , define

$$q = \log_2 \frac{L_{in} + 0.5L_{out}}{L_{in,0} + 0.5L_{out,0}}, \quad r = \log_2 \frac{k}{k_0}, \quad (2)$$

$$\phi = [q, r, qr, r^2]. \quad (3)$$

For metric m , ridge regression fits $\log(y_m / \hat{y}_m^{base}) = \beta_m^T \phi$. The ridge strength is selected per metric by leave-one-workload-out MAPE over the development set. We correct throughput, power, TTFT, and TPOT; energy is never independently fitted, but is recomputed from corrected power and duration. This keeps the correction sparse and the energy identity explicit.

B. Measured GPU Path

The L1 executor runs a digest-pinned vLLM server and CPU-only benchmark client in restricted Docker containers. Prefix caching is disabled; temperature, seed, and output length are fixed. A READY/GO/DONE handshake places DCGM cumulative-energy reads around the measured request window, excluding model loading, graph capture, and warmup. Each record retains 100-ms telemetry, stdout/stderr, model and tokenizer revisions, image digests, server controls, and power-limit readback. Repeated points are summarized by median and MAD, without discarding their raw traces. Docker fixes this measured path; it does not emulate GPU performance.

C. Scope and Release Gate

The CPU correction applies only to the exact measured model, runtime, serving configuration, output length, request regime, GPU count, and bounded context/concurrency envelope. Outside it, the executor abstains instead of extrapolating silently.

A second gate may narrow release inside that envelope: our confirmation campaign preregisters an occupancy-stratified TTFT rule, applies it without refitting, and writes the resulting pass/abstain decision to a hashed artifact. Even in scope, predictions can prioritize expensive runs but cannot certify themselves. Uncalibrated intervals and placeholder topology terms remain machine-labeled as requiring validation; a later random, Bayesian, or agentic policy cannot alter those evidence semantics.

IV. EVALUATION

A. Platform and Frozen Protocol

We test workload transfer on one NVIDIA H100 80GB at a 700-W limit (driver 580.173.02, DCGM 4.6.1). A digest-pinned vLLM 0.23.0 container serves the pinned Qwen2.5-7B-Instruct revision in BF16. The server uses TP=PP=DP=1, 256 maximum sequences, 8192 maximum batched tokens, chunked prefill, and disabled prefix caching. Every workload generates 128 output tokens at unbounded offered rate; request count is eight times concurrency.

The protocol has four chronological stages. First, 3 runs at 512 input tokens and concurrency 8 form the L1 anchor. Second, 6 workloads spanning 256–2,048 input tokens and concurrency 1–32, with three repeats each, fit the residual model; its leave-one-workload-out energy MAPE is 10.11%. Third, we freeze 8 disjoint context/concurrency predictions before collecting 24 cyclically balanced H100 measurements. The validator verifies chronology and 62 raw/configuration artifacts. Finally, without refitting, we preregister and freeze a second 3×3 grid: $L_{in} \in \{384, 768, 1536\}$ and $k \in \{1, 2, 4\}$. Its 27 runs and 61 artifacts test an all-workload energy MAPE threshold of 15% and a TTFT release boundary at $k = 4$.

B. Blind Holdout

Table I reports the first holdout. The correction lowers energy MAPE from 63.01% to 6.23%; median and maximum energy APE are 4.96% and 13.84%. Rank correlation is $\rho = 0.976$, and 27/28 workload pairs are correctly ordered. Throughput and TPOT improve to 10.53% and 7.99% MAPE. TTFT remains the weakest metric at 32.67%. At the worst sparse point, opposing power and duration errors leave energy within 13.77% while

TABLE I

INITIAL BLIND HOLDOUT: EIGHT UNSEEN WORKLOADS, THREE REAL H100 REPEATS EACH. BASE AND CORRECTED PREDICTIONS WERE FROZEN TOGETHER BEFORE MEASUREMENT. ORDER IS CORRECTLY RANKED PAIRS OUT OF 28 (NO TIES).

Metric	Base MAPE	Corrected MAPE	ρ	Order
Energy / 1k output	63.01%	6.23%	0.976	27/28
Throughput	23.93%	10.53%	0.976	27/28
P95 TTFT	73.44%	32.67%	0.952	26/28
P95 TPOT	34.29%	7.99%	0.929	25/28

TABLE II

PREREGISTERED SCOPE CONFIRMATION WITHOUT REFITTING. EACH ROW IS MAPE OVER THREE UNSEEN CONTEXTS AND THREE REAL H100 REPEATS PER CONTEXT.

k	Energy	Throughput	TPOT	TTFT	Release gate
1	5.78%	24.02%	18.25%	79.27%	TTFT abstain
2	9.15%	7.89%	6.90%	50.34%	TTFT abstain
4	7.12%	2.01%	1.94%	9.27%	pass
All	7.35%	11.31%	9.03%	46.29%	energy pass

TTFT misses by 138.8%; energy accuracy alone therefore cannot certify the decomposition.

C. No-Refit Scope Confirmation

The second holdout passes its preregistered primary endpoint: energy MAPE is 7.35%, maximum APE is 10.93%, $\rho = 0.933$, and 33/36 pairs are correctly ordered. The predicted and measured minimum-energy workload also match. This is replication with the unchanged v2 profile, not a model selected after viewing the first holdout.

The occupancy rule is equally decisive. At $k = 4$, TTFT MAPE is 9.27% and no point exceeds 14.55%, passing the 20% gate; throughput and TPOT MAPE are only 2.01% and 1.94%. Across $k \leq 2$, TTFT MAPE is 64.80% and the maximum APE is 124.36%, so the preregistered result is to withhold TTFT below four. Energy remains below 10% MAPE in every stratum, but the $k = 1$ throughput and TPOT errors (24.02% and 18.25%) again warn that this may partly reflect component-error cancellation.

D. Repeatability, Provenance, and Cost

Median energy repeat CV is 0.78% in the first holdout and 0.68% in the confirmation. All 51 blind runs succeeded; each prediction preceded measurement and every client output and DCGM trace passed its manifest hash. Excluding the three anchor runs, 69 development/holdout measurements consumed 0.404 H100-hours; v3 alone consumed 0.145 H100-hours. Frozen intervals contain 31/32 and 35/36 metric-workload medians in the two studies, but were not independently calibrated, so we make no statistical coverage claim. The final archive digests are 2d741792...12ecafb and 9a4dbfa5...22cd3fd.

V. ARTIFACT AND REPRODUCIBILITY

Executor separation. The same candidate schema is consumed by a CPU-resident projector and a measured executor.

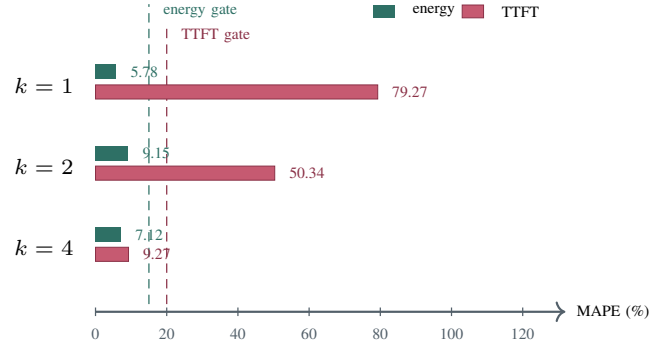


Fig. 2. The no-refit confirmation passes its all-workload 15% energy-MAPE endpoint, with every concurrency stratum below 15%. TTFT passes its 20% gate at $k = 4$ but fails below it, triggering explicit abstention.

The former emits deterministic simulated or extrapolated JSONL with a decomposition and scope decision, without allocating a target GPU. The latter controls the real server, benchmark client, power limit, DCGM sampler, and measurement boundary, then emits L1 evidence. Prediction code cannot relabel its own record as measured. A campaign runner expands the frozen workload list, applies a cyclically balanced schedule, checkpoints every trial, and resumes only exact workload/repeat matches.

Implemented workflow. Separate CLI commands build an L1 profile, fit development-only residuals, freeze predictions, execute measured trials, validate campaigns, and apply preregistered release rules. Fitting accepts only a verified validation report; holdout records are rejected as training input. Residual reuse requires exact model, serving configuration, output length, offered rate, GPU count, and in-range workload features. Tests cover scope decisions, artifact tampering, evidence labels, and holdout leakage.

Isolation and archive. The CPU client reaches vLLM only through an internal Docker bridge; containers use read-only filesystems, dropped Linux capabilities, process limits, and pinned image/model revisions. GPU power control stays on the host. These restrictions make the reviewed workload reproducible; they do not make arbitrary generated code safe. The two sealed archives contain frozen predictions, 51 measurements, DCGM traces, stdout/stderr, server metadata, run order, reports, decisions, and SHA-256 manifests. Both archives and directly readable reports are published with the repository; their digests and measurement/analysis commits are recorded.

CPU projection overhead remains to be reported as time and peak memory, not treated as free. The next stage is configuration ranking under fixed workloads, using energy rank, SLO false-safe rate, Pareto recall, and measured GPU-hours avoided. TP/PP and H100/H200/B200 transfer require sparse target-topology anchors and independent target reruns.

VI. FROM PREDICTION TO DEPLOYMENT DECISIONS

The two holdouts establish workload screening under one fixed configuration, not configuration selection. A deployment

TABLE III

CLAIM LADDER. COMPLETED WORKLOAD EVIDENCE CANNOT BE REUSED AS PROOF OF CONFIGURATION, TOPOLOGY, OR HARDWARE TRANSFER.

Claim	Real evidence	Gate
Measurement path	3-run anchor + raw traces	Boundaries + hashes: done
Workload transfer	6×3 dev + (8+9)×3 blind	Energy replicated; TTFT scoped
Serving controls	Batch/cache grid	SLO safety, Pareto recall
TP/PP topology	Sparse scale probes + reruns	Rank and false-safe rate
Hardware transfer	H100/H200/B200 anchors	Cold-start cost and target error

search must pass three further gates. **Ranking** requires top- k recovery, Pareto precision/recall, and hypervolume error on a frozen candidate set. **SLO safety** reports both the rate of points predicted safe but measured unsafe and whether the final recommendation is false-safe; intervals must be calibrated before a disjoint holdout. Current diagnostic containment is not such a guarantee. **Cost** charges every calibration, warmup, failed job, and verification as $C_{GPU}(D) = \sum_{e \in D} g_e t_e / 3600$ GPU-hours. CPU screening is reported separately in CPU-seconds and peak memory: zero target GPU-hours is not zero energy. Success requires matched SLO safety with fewer purchased GPU-hours after disclosing this overhead. Table III separates completed evidence from these future claims.

VII. RELATED WORK AND POSITIONING

Inference-energy measurement. Inference energy depends on token geometry, model, framework, and hardware [1], [2]. ML.ENERGY automates measurement across systems, while TokenPowerBench contributes declarative workloads and phase-aligned device/node telemetry [3], [4]. These systems answer what a completed run consumed. TokenPowerSandbox retains real-GPU measurement as calibration source and final authority, while CPU-resident simulated or extrapolated records estimate which runs to purchase next. Its phase boundary preserves the semantics needed to compare them.

Serving simulation and recommendation. Vidur provides event-driven performance simulation for LLM serving; LLM-Pilot characterizes models and hardware to recommend deployments; and Charon models fine-grained large-scale training and inference [5], [6], [7]. Our current model is deliberately local: an analytical decomposition supplies a prior, six measured workloads learn only a sparse residual, and two no-refit holdouts audit its release boundary. An exact configuration contract bounds reuse. It neither creates virtual GPUs nor claims that one H100 measurement reproduces communication, queueing, or power behavior on a multi-node target. Its differentiator is the release boundary: split labels, frozen hashes, abstention, and an independent run are first-class outputs, not evaluation bookkeeping added after a recommendation.

Energy-aware serving optimization. DynamoLLM designs inference clusters under performance and energy objectives; BEAM jointly tunes resources and power under SLOs; and BOute applies multi-objective Bayesian optimization to heterogeneous LLM serving [8], [9], [10]. These systems are closest to the eventual deployment use case, but solve a different layer: a policy searches a configuration space given an evaluator. This WIP audits the evaluator itself. It therefore reports blind point error, ordering, false-confidence risks, and purchased measurement cost before claiming energy savings. A future TokenPowerAgent can compare random, Bayesian, and agentic policies over the same evidence API; no policy will be allowed to promote a predicted record to measured evidence or train on its final holdout. The present result therefore supports same-GPU workload screening within a measured envelope; configuration, topology, and hardware transfer remain separate experiments in the claim ladder.

VIII. LIMITATIONS AND CONCLUSION

The evidence covers one model, one H100, one fixed serving configuration, fixed 128-token outputs, and workload transfer only. It does not establish configuration ranking, finite-rate queueing, quality equivalence, power-cap transfer, TP/PP fidelity, H200/B200 transfer, or net deployment energy savings. Six development workloads are small for a four-feature model. The preregistered low-concurrency TTFT failure shows that a simple range envelope is too permissive near sparse corners, while the resulting $k < 4$ abstention is validated only for this workload family. DCGM measures GPU energy but excludes host, network, cooling, and embodied costs. The current prediction intervals are engineering bounds, not calibrated confidence intervals. CPU projection energy and queue-time savings are also unmeasured, so “zero-GPU” denotes resource allocation, not net energy.

Within that boundary, TokenPowerSandbox reaches 6.23% and 7.35% energy MAPE on two sealed holdouts with no intervening refit. The second study also converts a preregistered latency failure into an explicit abstention rather than a confident recommendation. The central result is not that a Docker container simulates an H100. It is that CPU-first screening can guide scarce GPU experiments only when prediction, provenance, scope decisions, and real validation are one system. The next step is to test whether this contract reduces measured GPU-hours for SLO-constrained configuration and topology search on HPC clusters. **AI-assistance disclosure.** OpenAI Codex assisted with literature organization, language editing, LaTeX drafting, and consistency checks. The authors verified the manuscript, citations, code, and results; AI is not an author.

REFERENCES

- [1] S. Samsi, D. Zhao, J. McDonald, B. Li, A. Michaleas, M. Jones, W. Bergeron, J. Kepner, D. Tiwari, and V. Gadepally, "From words to watts: Benchmarking the energy costs of large language model inference," in *2023 IEEE High Performance Extreme Computing Conference*, 2023, pp. 1–9.
- [2] J. Fernandez, C. Na, V. Tiwari, Y. Bisk, S. Luccioni, and E. Strubell, "Energy considerations of large language model inference and efficiency optimizations," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 32 556–32 569.
- [3] J.-W. Chung, J. J. Ma, R. Wu, J. Liu, O. J. Kweon, Y. Xia, Z. Wu, and M. Chowdhury, "The ML.ENERGY benchmark: Toward automated inference energy measurement and optimization," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [4] C. Niu, W. Zhang, J. Li, Y. Zhao, T. Wang, X. Wang, and Y. Chen, "TokenPowerBench: Benchmarking the power consumption of LLM inference," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, 2026, pp. 32 582–32 590.
- [5] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov, "Vidur: A large-scale simulation framework for LLM inference," in *Proceedings of Machine Learning and Systems*, vol. 6, 2024.
- [6] M. Łazuka, A. Anghel, and T. Parnell, "LLM-Pilot: Characterize and optimize performance of your LLM inference services," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024.
- [7] M. Yang, Z. Zhang, M. Wu, J. Yan, H. Sun, and L.-W. Chang, "Charon: A unified and fine-grained simulator for large-scale LLM training and inference," in *Proceedings of Machine Learning and Systems*, vol. 8, 2026.
- [8] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, "DynamoLLM: Designing LLM inference clusters for performance and energy efficiency," in *2025 IEEE International Symposium on High Performance Computer Architecture*, 2025, pp. 1348–1362.
- [9] H. Lee, S. Choi, S. Lim, and Y. Kwon, "BEAM: Joint resource–power optimization for energy-efficient LLM inference under SLO constraints," in *Proceedings of Machine Learning and Systems*, vol. 8, 2026.
- [10] Y. Jiang, F. Fu, and E. Yoneki, "BOute: Cost-efficient LLM serving with heterogeneous LLMs and GPUs via multi-objective bayesian optimization," in *Proceedings of Machine Learning and Systems*, vol. 8, 2026.